Project Title: DataClinic — Data Quality & Pipeline Observability Platform

Program: DEPI — Digital Egypt Pioneers Initiative

Instructor: Eng. Mohamed Shehab

Team (6 Members): [Abrar Mahmoud] · [Mohamed Tharwat] · [Sohaila Gaber] · [Abanob Ashraf] · [Shimma Ebrahim] · [Alaa Elrefae]

Duration: 9 Weeks.

1. Project Planning & Management

1.1 Project Proposal — Summary:

Field	Content
Overview	DataClinic is a production-style data engineering platform simulating a healthcare cloud computing company that stores and manages patient data on behalf of partner hospitals. Hospitals push daily batches of patient records, visits, and lab results. DataClinic validates every batch against a data-quality framework, routes invalid records to a dead-letter table, orchestrates the ETL with Apache Airflow, and monitors pipeline health through Prometheus metrics, Grafana dashboards, and alerts.
Problem Statement	Data pipelines commonly fail silently — bad data is discovered only after business users report broken dashboards. In healthcare, this delay is unacceptable: incomplete patient data can affect clinical decisions. DataClinic shifts issue detection from *end users* to *engineers*, before data reaches reporting layers.
Objectives	1) Build a representative ETL pipeline for hospital data feeds · 2) Implement a data-quality framework (6 checks) · 3) Add reliability: exception handling, retries, dead-letter records, execution logging · 4) Expose operational metrics via Prometheus · 5) Build a Grafana dashboard with alert rules · 6) Containerize with Docker Compose + automated testing (GitHub Actions)
Deliverables	Working ETL pipeline · Airflow DAG · DQ framework · Dead-letter/rejection tables · Prometheus configuration · Grafana dashboard · Alert examples · Failure-recovery demonstration · Documentation · Containerized environment

1.2 Project Plan — Scope:

In Scope	Out of Scope
ETL pipeline + synthetic hospital data generator	Real HIPAA compliance certification
Data-quality framework (6 checks)	HL7 / FHIR / DICOM medical interfaces
Airflow orchestration, retries, dead-letter handling	End-user web application
Prometheus metrics, Grafana dashboard, alerts	Real hospital integrations
Docker Compose, CI (GitHub Actions), documentation	

1.3 Project Plan — 9-Week Timeline

WeekPhase / Activities		Deliverable
1	Kickoff, requirements gathering, stakeholder analysis, architecture design	Requirements document + architecture diagram
2	Docker Compose environment (Postgres, Airflow, Prometheus, Grafana) + synthetic data generator	Working environment + data generator
3	Core ETL pipeline v1 (extract → stage → load)	First runnable pipeline
4	Data-quality framework (all six checks) + unit tests	DQ module
5	Dead-letter tables, rejection reports, execution logging	Error-handling layer
6	Airflow DAG: orchestration, retries, failure handling	Production DAG

Week	Phase / Activities	Deliverable
7	Prometheus metrics, Grafana dashboard, alert rules	Complete monitoring stack
8	Integration testing, failure-recovery scenarios, GitHub Actions CI	Demo scripts + CI pipeline
9	Final documentation, presentation rehearsal, live demo, submission	Final delivery

Milestones:

Milestone	Deadline	Definition of Done
M1 — Design Approved	End of Week 1	Requirements + architecture approved by instructor
M2 — Pipeline v1 Running	End of Week 3	Data flows end-to-end in the environment
M3 — Quality Layer Complete	End of Week 5	6 DQ checks + dead-letter handling working
M4 — Full Stack Operational	End of Week 7	Orchestration + monitoring + alerts live
M5 — Final Delivery	End of Week 9	Demo + documentation + submission

Gantt Chart:

Planning, Requirements & Design

Environment & Data Generator

Core ETL Pipeline

DQ Framework

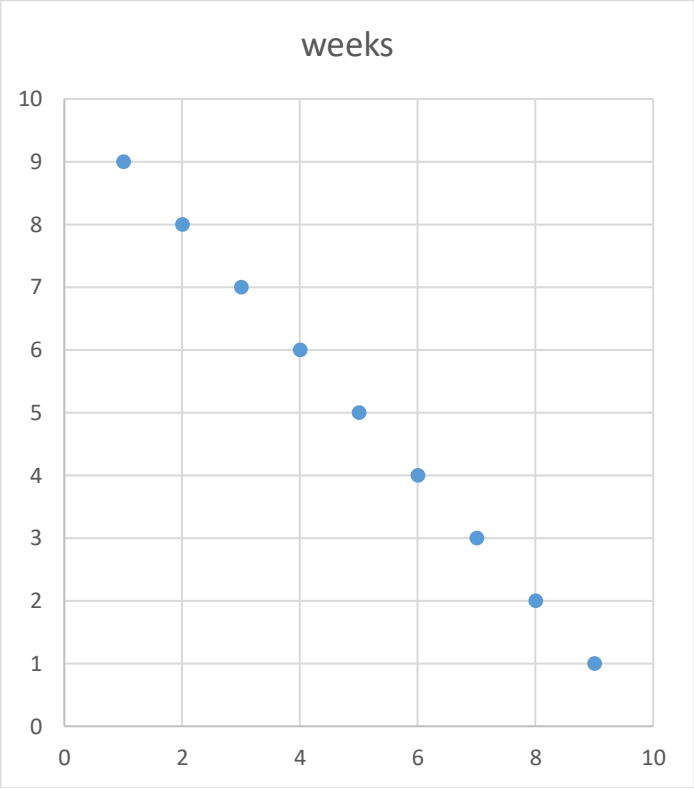
Error Handling & Logging

Airflow Orchestration

Monitoring & Alerting

Testing, CI & Recovery Demo

Docs, Presentation & Delivery



1.4 Resource Allocation

Resource	**Description**
Human	6 team members (roles in §1.5), weekly sync meetings
Hardware	Personal laptops (8 GB RAM minimum recommended)
Software	Python, PostgreSQL, Apache Airflow, Prometheus, Grafana, Docker Compose, Git/GitHub — all free & open source
Budget	Zero budget; open-source stack only

1.5 Task Assignment & Roles (6 Members)

Member	Role	Key Responsibilities
[Abrar Mahmoud]	Team Lead & Data Engineer	Overall coordination & planning, ETL pipeline design & implementation, code reviews, final integration
[Mohamed Tharwat]	Data Quality Engineer	Data-quality framework (6 checks), validation rules engine, DQ unit tests
[Sohaila Gaber]	Data Engineer — Ingestion	Synthetic hospital data generator, staging layer, dead-letter/rejection handling
[Abanob Ashraf]	DevOps / Orchestration Engineer	Docker Compose environment, Airflow DAGs, retries & failure handling, GitHub Actions CI
[Shimma Ebrahim]	Observability Engineer	Prometheus metrics, Grafana dashboards, alert rules, execution logging
[Alaa Elrefae]	QA & Documentation Lead	Test plans, failure-recovery demo scenarios, documentation, presentation coordination

Shared responsibilities: presentation preparation, final live demo, README review.

1.6 Risk Assessment & Mitigation Plan

#	Risk	Likelihood	Impact	Mitigation Strategy
1	Airflow setup complexity inside Docker	Medium	High	Use official Airflow Docker Compose file; pin versions; allocate full Week 2; fallback to local install
2	Compressed 9-week schedule	High	Medium	Parallel workstreams per role; weekly progress checkpoints; MVP-first prioritization
3	Integration issues between components	Medium	Medium	Weekly integration checkpoints; version pinning; containerized consistent environment
4	Team member unavailability (exams, illness)	Medium	Medium	All tasks tracked as GitHub issues; shared documentation; buddy system on critical tasks
5	Scope creep (extra medical formats, extra features)	Medium	Medium	Freeze scope after Week 1; change requests go through team lead
6	Unrealistic/insufficient test data	Low	Low	Configurable generator with deliberate "dirty data" injection
7	Learning curve on Prometheus/Grafana	Medium	Low	Tutorials in Weeks 1–2; pair programming on observability tasks

1.7 Key Performance Indicators (KPIs)

System KPIs:

KPI	Target
Pipeline run success rate	≥ 95% of scheduled runs succeed
Data-quality check coverage	100% of the 6 required checks implemented and tested
Rejected-record handling	100% of invalid records stored in dead-letter table with a reason
Alert latency	Failed/abnormal run triggers an alert within 1 minute
Silent failures	Zero — every failure is logged, metriced, and alerted
Processing time	Standard 10K-row batch processed in ≤ 5 minutes
Test coverage (DQ framework)	≥ 80%
Dashboard responsiveness	Dashboard loads in < 3 seconds

Project Management KPIs:

KPI	Target
Milestone adherence	100% of milestones (M1–M5) delivered on schedule
Task completion rate	≥ 90% of GitHub issues closed per week
Documentation completeness	All deliverables documented before submission

2. Literature Review

2.1 Review of Existing Tools & Approaches

Tool / Platform	Type	Key Idea	Relation to DataClinic
Great Expectations	Open-source DQ library	"Expectations as code" — declarative validation	Inspired our check design; we build a lightweight custom version to understand internals
dbt tests	SQL-based testing framework	Assertions on models inside the transformation layer	Shows DQ placement inside transformation; ours runs pre-load
Soda Core	Open-source DQ as code	Checks defined in a simple DSL, results as metrics	Similar check taxonomy (nulls, schema, freshness)
AWS Deequ	DQ library for Spark	Data-quality metrics on large-scale data	Validates the "metrics from DQ checks" approach we adopt
Apache Griffin	DQ platform	Centralized DQ service for big data	Example of DQ as a platform rather than ad-hoc scripts
Monte Carlo / Bigeye	Commercial data observability	Monitoring via metrics, anomaly detection, alerting	Direct model for our observability layer
Prometheus + Grafana	Monitoring stack (industry standard)	Time-series metrics + visualization + alerting	The exact stack used in our implementation

Conclusion: Industry practice combines three layers — orchestration (Airflow), data-quality validation (Great Expectations/Soda-style), and observability (Prometheus/Grafana). DataClinic intentionally implements all three in a lightweight, production-style architecture, using a custom DQ framework to demonstrate deep understanding rather than importing a black-box library.

2.2 Feedback & Evaluation *(to be completed by the instructor)*

Criterion	Instructor Assessment
Proposal clarity & relevance	
Technical depth	
Documentation quality	

2.3 Suggested Improvements *(to be completed after feedback)*

#	Suggested Improvement	Status
1		☐ Planned / ☐ Implemented

2.4 Final Grading Criteria

Component	Weight	Notes
Documentation	[20]%	Proposal, plan, requirements, final report
Implementation	[40]%	Pipeline, DQ framework, monitoring stack
Testing	[15]%	Unit + integration tests, CI
Presentation & Demo	[25]%	Live failure-recovery demonstration

3. Requirements Gathering:

3.1 Stakeholder Analysis:

Stakeholder	Role in System	Needs & Concerns
Data Engineering Team	Operates the pipelines	Reliable runs, automatic retries, clear failure reasons
Data Quality Lead	Owns data standards	Enforced checks on every batch; rejection reports per batch
Hospital Data Officers *(clients)*	Send daily data feeds	Confirmation of accepted/rejected counts; actionable feedback on format errors
Compliance / Governance Officer	Audit & security	Full run logging; zero silent data loss; no PHI in logs
Analytics / Business Teams	Consume reporting dashboards	Dashboards fed only by validated data; trustworthy numbers
SRE / DevOps Team	System health	Live metrics, dashboards, and alerts for failed or abnormal runs
DEPI Instructor (Eng. Mohamed Shehab)	Supervision & evaluation	Clear milestones, working demo, complete documentation

3.2 User Stories

ID	User Story
US-01	As a data engineer, I want failed tasks to retry automatically so that transient issues don't require manual intervention.
US-02	As a data engineer, I want invalid records routed to a dead-letter table with a reason so I can investigate without blocking the pipeline.
US-03	As a data quality lead, I want six categories of checks enforced on every batch so bad data never reaches analytics tables.
US-04	As a hospital data officer, I want a per-batch summary (accepted vs. rejected) so I can fix upstream issues at the source.
US-05	As an SRE, I want Prometheus metrics and a Grafana dashboard so I can see pipeline health at a glance.
US-06	As an SRE, I want alerts on failures or abnormal duration so I can respond immediately.
US-07	As a compliance officer, I want every run logged (start, end, status, counts) so audits are possible.
US-08	As an analyst, I want dashboards fed only from validated data so I can trust the reports.

3.3 Use Cases:

UC-01 — Process a Daily Hospital Batch (Main Use Case):

Field	Description
Actor	Airflow Scheduler (system)
Precondition	Hospital batch file available in the landing area
Main Flow	1. DAG triggers on schedule → 2. Extract batch → 3. Run 6 DQ checks → 4. Load valid records to curated tables → 5. Send invalid records to dead-letter table with reason codes → 6. Write run log → 7. Publish metrics to Prometheus
Alternative Flows	• File missing → task fails → retry → alert if retries exhausted • DQ rejection rate exceeds threshold → quality alert raised
Postcondition	Curated tables updated; rejection report available; metrics & logs updated

UC-02 — Handle a Failed Pipeline Run:

Field	Description
Actor	Data Engineer / SRE
Trigger	Task failure detected by Airflow
Main Flow	1. Airflow retries task (3 attempts, backoff) → 2. On final failure: status = failed, failure logged → 3. Alert fired → 4. Engineer inspects dashboard + logs → 5. Root cause fixed → next scheduled run recovers
Postcondition	Failure documented in run history; recovery demonstrated

UC-03 — Monitor Pipeline Health:

Field	Description
Actor	SRE
Main Flow	Opens Grafana dashboard → views run duration, success/failure trend, records processed vs. rejected, DQ failures by check type

Other Use Cases (Summary)

ID	Use Case	Actor
UC-04	Respond to a data-quality alert	SRE / DQ Lead
UC-05	Generate a per-batch rejection report	Data Engineer
UC-06	Review execution logs for audit	Compliance Officer

3.4 Functional Requirements

ID	Requirement
FR-01	The system shall ingest daily batch files (CSV) containing patients, visits, and lab results.
FR-02	The system shall validate null values in mandatory columns.
FR-03	The system shall detect duplicate records on business keys (e.g., patient_id, visit_id).

ID	Requirement
FR-04	The system shall perform schema/type validation (column names and data types).
FR-05	The system shall perform row-count validation (minimum threshold / expected count per batch).
FR-06	The system shall enforce referential integrity (e.g., every lab_result.patient_id must exist in patients).
FR-07	The system shall enforce range validation (e.g., age 0–120, heart rate 30–220).
FR-08	Valid records shall be loaded to curated tables; rejected records shall be stored in a dead-letter table with rejection reason, source batch, and timestamp.
FR-09	The pipeline shall be orchestrated by an Airflow DAG with automatic retries (3 attempts, exponential backoff).
FR-10	Failed tasks shall be marked, logged, and trigger an alert after retries are exhausted.
FR-11	The system shall produce structured execution logs per run and per task.
FR-12	The system shall expose Prometheus metrics: run duration, success/failure counters, records processed, records rejected, DQ failures by check type.
FR-13	A Grafana dashboard shall visualize all metrics from FR-12.
FR-14	Alert rules shall exist for: pipeline failure, high rejection rate, and abnormal run duration.

ID	Requirement
FR-15	The system shall generate a rejection report summarizing each batch.
FR-16	The full stack shall start with a single Docker Compose command.
FR-17	*(Extension)* GitHub Actions shall run the test suite on every push.

3.5 Non-Functional Requirements:

ID	Category	Requirement
NFR-01	Performance	Standard 10K-row batch processed end-to-end in ≤ 5 minutes.
NFR-02	Reliability	Automatic retry mechanism; ≥ 95% scheduled-run success rate; zero silent failures.
NFR-03	Observability	Every run traceable via logs + metrics; alert latency < 1 minute.
NFR-04	Security (simulated)	Credentials via environment variables; no patient-identifying data in logs; services on an internal Docker network only.
NFR-05	Usability	One-command startup; dashboard interpretable within 1 minute; complete setup documentation.
NFR-06	Maintainability	Modular DQ framework — new checks added without modifying pipeline core; documented code style; test suite.

ID	Category	Requirement
NFR-07	Portability	Runs on any machine with Docker Compose; no host-specific configuration.
NFR-08	Auditability	Full run history and rejection history queryable in the database.